

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Annexure A Constraint-Based Problem Solving

Code of Conduct:

M. Phanith Chandu (192425238) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Phanith Chandu

Q. No	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)					Total Marks (Out of 50)	

Annexure A

Constraint-Based Problem Solving

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an **OCR system** that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect **facial expressions (happy, sad, neutral)** with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

1. Real-Time Face Recognition System (Edge Deployment)

Design a face recognition system for a low-power edge device such as Raspberry Pi. It must process live video in real time, tolerate varying lighting, use limited CPU/memory, and recognize people from a predefined dataset.

- Real-time operation: keep processing latency low enough for live video (targeting below the stated 30-FPS latency requirement).
- Varying illumination: faces may be bright, dark, backlit, or unevenly illuminated.
- Limited resources: Raspberry Pi has less CPU, memory, and power than a desktop computer.
- Known identities: recognition is limited to a predefined set of enrolled people.

Priority analysis: The highest priorities are computational efficiency and stable face detection. A lightweight detector should run on resized frames, while recognition should operate only on detected face regions rather than the complete frame. Illumination normalization should be applied without expensive processing. A small face database can be kept in memory.

1. Image acquisition – capture frames using `cv2.VideoCapture()` from a USB camera or Pi camera.
2. Frame reduction – resize the frame to a smaller working resolution to reduce CPU load.
3. Preprocessing – convert BGR to grayscale and apply histogram equalization or CLAHE to improve robustness to lighting changes.
4. Face detection – use a lightweight OpenCV Haar/LBP cascade or another compact detector to locate faces.

OpenCV VideoCapture, resizing, grayscale conversion, CLAHE/histogram equalization, Haar/ LBP cascade detection, LBPH face recognition, ROI processing, and `cv2.rectangle/cv2.putText`.

Justification

Resizing reduces the number of pixels processed per frame and therefore lowers CPU usage. Grayscale and local illumination normalization are inexpensive. Detecting first and recognizing only the face ROI avoids unnecessary computation. LBPH is suitable for small, predefined datasets and is lighter than a large deep-learning model. Processing every frame or using frame skipping can be adjusted according to measured FPS on the edge device.

End-to-End Flow: Camera → Resize → Illumination normalization → Face detection → Face ROI → LBPH recognition → Label/Output

2. Automated Video Surveillance with Alert System

A. Problem Understanding (25%)

Create an OpenCV surveillance system that detects motion and unusual activity from standard CCTV feeds, generates real-time alerts, and minimizes false positives in environments containing noise and moving backgrounds.

- Motion and unusual-activity detection must work continuously.
- Alerts must be generated with low delay.
- The input should support standard CCTV/video streams.
- False positives must be reduced in dynamic environments.
- Noise, shadows, camera vibration, and moving background objects must be handled.

Priority analysis: False-positive reduction is the key constraint. Simple frame differencing is fast but sensitive to noise. Background subtraction is more suitable, but the model must adapt to gradual scene changes. Motion should be confirmed over multiple frames and inside meaningful regions before an alert is produced.

1. Video acquisition – read CCTV frames using `cv2.VideoCapture()`, including an IP/RTSP source when supported by the deployment setup.
2. Preprocessing – resize frames, convert to grayscale when appropriate, and apply Gaussian blur to reduce sensor noise.
3. Motion extraction – use `cv2.createBackgroundSubtractorMOG2()` or KNN to generate a foreground mask.
4. Mask cleanup – apply morphological opening/closing with `cv2.morphologyEx()` to remove small noise and fill gaps.

unding boxes, status text, timestamps, and alert indicators using OpenCV.

VideoCapture, GaussianBlur, MOG2/KNN background subtraction, thresholding, morphology, contour detection, ROI filtering, temporal confirmation, and OpenCV drawing functions.

Justification

Background subtraction is computationally practical for CCTV and separates moving foreground from a mostly stationary scene. Gaussian blur and morphology suppress noise. Area filtering prevents tiny changes from causing alerts, while temporal confirmation prevents one-frame disturbances from being reported. Adaptive background models help accommodate gradual changes in the scene.

End-to-End Flow: CCTV → Preprocess → Background subtraction → Noise removal → Contours → Activity filtering → Temporal confirmation → Alert

3. OCR System for Low-Quality Documents

Develop an OpenCV preprocessing pipeline that improves OCR of low-resolution, noisy scanned documents with varying fonts and illumination, while remaining efficient for batch processing.

- Input documents may be low-resolution and noisy.
- Illumination may be uneven.
- Fonts and character shapes can vary.
- Text and background must be separated reliably.
- Batch processing requires efficient and repeatable operations.

Priority analysis: The main priority is to improve character separation before OCR. The pipeline should remove noise without destroying character strokes, correct illumination, enhance contrast, and create a clean binary text image. Expensive processing should be avoided when simple OpenCV operations are sufficient.

1. Image loading – read each scanned image with `cv2.imread()`.
2. Grayscale conversion – use `cv2.cvtColor()` to reduce processing complexity.

3. Noise reduction – apply `cv2.GaussianBlur()` or median filtering while preserving character structure.
4. Illumination correction – use adaptive thresholding or local normalization to handle uneven lighting.

`imread`, `cvtColor`, `GaussianBlur`/median filtering, CLAHE, Otsu thresholding, adaptive thresholding, morphology, contour/segmentation operations, resizing, and batch file processing.

Justification

Grayscale reduces computation, while blur removes high-frequency noise. Adaptive thresholding is particularly useful when illumination is not uniform. Morphological operations improve the continuity of characters and remove isolated noise. Standardized preprocessing makes different documents more consistent, improving downstream OCR. The same lightweight sequence can be applied repeatedly to a batch.

End-to-End Flow: Scanned image → Grayscale → Denoise → Contrast/illumination correction → Threshold → Morphology → OCR → Text

4. Facial Expression Recognition for Mobile Application

A. Problem Understanding (25%)

Design a mobile-friendly system that detects faces and classifies expressions as happy, sad, or neutral while using limited CPU, operating in real time, and tolerating slight head movement and lighting changes.

- Limited mobile CPU and memory.
- Real-time response is required.
- Only three expression classes are needed: happy, sad, and neutral.
- Small head movements should not immediately break detection.
- Lighting changes should have limited effect.

Priority analysis: The solution should avoid processing the full image with a heavy model. Face detection can be performed at reduced resolution, followed by ROI extraction. Expression classification should use a lightweight classifier trained on normalized face images. Temporal smoothing can stabilize predictions when the head moves slightly or one frame is uncertain.

1. Camera acquisition – capture frames using the mobile camera interface or an OpenCV-compatible stream.
2. Frame resizing – reduce resolution before detection to save CPU.
3. Face detection – use a lightweight Haar/LBP detector or compact face detector.
4. ROI extraction – crop the detected face and resize it to a fixed input size.

Face detection, ROI processing, grayscale conversion, CLAHE/equalization, feature extraction/classification, temporal smoothing, resizing, and OpenCV visualization.

Justification

Reduced-resolution detection lowers CPU usage. Processing only the face ROI saves additional computation. Illumination normalization makes facial features more consistent. A compact classifier is more appropriate for a mobile CPU than a large network. Temporal smoothing improves stability during slight head movement without requiring expensive processing.

End-to-End Flow: Camera → Resize → Face detection → Face ROI → Normalize → Expression classifier → Temporal smoothing → Label

5. Smart Traffic Monitoring System

Develop an OpenCV system that detects vehicles from live video, tracks their movement across frames, counts vehicles, displays statistics, and operates continuously with minimal delay.

- Vehicles must be detected from live video.
- Vehicle identities/positions should be associated across consecutive frames.
- Vehicles must be counted without repeatedly counting the same vehicle.
- Statistics should be displayed continuously.
- The system must remain efficient for long-running real-time operation.

Priority analysis: The most important issue is avoiding duplicate counts. Detection alone cannot reliably count vehicles because the same vehicle appears in many frames. A tracking stage should associate detections across frames, and a virtual counting line/zone should increment the count only when a tracked vehicle crosses it in the intended direction. Resolution and detection frequency should be balanced against real-time performance.

1. Video acquisition – read live traffic video with `cv2.VideoCapture()`.
2. Preprocessing – resize frames and optionally blur them to reduce noise.
3. Vehicle detection – use background subtraction such as MOG2/KNN for a fixed camera, followed by thresholding and morphology; alternatively use a compact object detector when available.
4. Contour extraction – find vehicle regions using `cv2.findContours()` and filter detections by area and shape.
5. Tracking – assign an ID to each detected vehicle and update its centroid/position across frames using a lightweight centroid or tracker-based approach.
6. Counting – draw a virtual line/zone and increment the counter when a tracked vehicle crosses it in the required direction.

7. Visualization – draw bounding boxes, IDs, counting line, vehicle count, FPS, and other statistics using `cv2.rectangle()`, `cv2.line()`, and `cv2.putText()`.

8. Continuous operation – release resources correctly and optimize frame size/detection frequency for stable real-time processing.

VideoCapture, background subtraction, thresholding, morphology, contour detection, centroid tracking, line-crossing logic, FPS measurement, and OpenCV drawing functions.

Justification

For a fixed CCTV camera, background subtraction can be much lighter than running a heavy detector on every frame. Morphological filtering and area thresholds remove noise. Tracking prevents the same vehicle from being counted repeatedly. A virtual line provides a simple and explainable counting rule. Resizing and controlled detection frequency reduce processing load while preserving enough information for traffic monitoring.

End-to-End Flow: Traffic video → Preprocess → Vehicle detection → Contours/objects → Tracking → Line crossing → Count → Statistics